

Remote Development Project Report

Student Name: Your Name

Student ID: Your Student ID

1. System Configuration

This assignment was completed in an Ubuntu virtual machine environment. I used Linux command-line tools to check the CPU information, memory size, operating system version, GCC compiler version, and Python version.

The commands used were:

```
lscpu
free -h
uname -a
gcc --version
python3 --version
```

The collected system information is shown in Appendix A.

2. Implementation Details

This project implements matrix multiplication using Python.

For two matrices A and B, the element in row i and column j of the result matrix C is calculated by multiplying the corresponding elements from row i of matrix A and column j of matrix B, then summing the products.

The formula is:

$$C[i][j] = A[i][0] * B[0][j] + A[i][1] * B[1][j] + \dots + A[i][k] * B[k][j]$$

The program uses three nested loops:

1. The first loop iterates through the rows of matrix A.
2. The second loop iterates through the columns of matrix B.
3. The third loop computes the dot product for each position in the result matrix.

3. Python Implementation

The source code file is named `matmul.py`.

The program can be executed with the following command:

```
python3 matmul.py
```

The program contains three main parts:

1. `matmul(A, B)`: implements matrix multiplication.
2. `verify()`: verifies correctness using a small example.

3. `performance_test(size)`: records the execution time for different matrix sizes.

4. Correctness Verification

To verify the correctness of the algorithm, I used a small example that can be checked manually.

```
A = [[1, 2],  
      [3, 4]]
```

```
B = [[5, 6],  
      [7, 8]]
```

According to the rule of matrix multiplication:

```
C[0][0] = 1*5 + 2*7 = 19  
C[0][1] = 1*6 + 2*8 = 22  
C[1][0] = 3*5 + 4*7 = 43  
C[1][1] = 3*6 + 4*8 = 50
```

Therefore, the expected result is:

```
[[19, 22],  
 [43, 50]]
```

The program output matches the expected result, so the implementation is correct.

5. Running Result

The output of running the Python program is shown in Appendix B.

6. Conclusion

Through this assignment, I practiced basic Linux command-line operations, including creating directories, checking system information, writing files, and running programs from the terminal.

I also implemented matrix multiplication in Python and verified the correctness of the algorithm using a manually calculated example. This assignment helped me understand how command-line tools, Markdown documentation, and Python programming can be combined in a simple remote development workflow.

7. References

1. Course slides: Lecture 3, UNIX/Linux environment and Shell.
2. Course slides: Lecture 4, Command Line Environment and Remote Development.
3. Python official documentation.

4. GNU GCC documentation.

Appendix A. System Information

==== CPU Information =====

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Address sizes:	45 bits physical, 48 bits virtual
Byte Order:	Little Endian
CPU(s):	2
On-line CPU(s) list:	0,1
Vendor ID:	GenuineIntel
Model name:	Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
CPU family:	6
Model:	165
Thread(s) per core:	1
Core(s) per socket:	1
Socket(s):	2
Stepping:	2
BogoMIPS:	4992.00
Flags:	fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
Hypervisor vendor:	VMware
Virtualization type:	full
L1d cache:	64 KiB (2 instances)
L1i cache:	64 KiB (2 instances)
L2 cache:	512 KiB (2 instances)
L3 cache:	16 MiB (2 instances)
NUMA node(s):	1
NUMA node0 CPU(s):	0,1
Vulnerability Gather data sampling:	Unknown: Dependent on hypervisor status
Vulnerability Ghostwrite:	Not affected
Vulnerability Indirect target selection:	Mitigation; Aligned branch/return thunks
Vulnerability Itlb multihit:	KVM: Mitigation: VMX unsupported
Vulnerability L1tf:	Not affected
Vulnerability Mds:	Not affected
Vulnerability Meltdown:	Not affected
Vulnerability Mmio stale data:	Vulnerable: Clear CPU buffers attempted, no microcode
Vulnerability Old microcode:	Not affected
Vulnerability Reg file data sampling:	Not affected
Vulnerability Retbleed:	Mitigation; Enhanced IBRS
Vulnerability Spec rstack overflow:	Not affected
Vulnerability Spec store bypass:	Mitigation; Speculative Store Bypass disabled via p
Vulnerability Spectre v1:	Mitigation; usercopy/swapgs barriers and __user po
Vulnerability Spectre v2:	Mitigation; Enhanced / Automatic IBRS; IBPB condit
Vulnerability Srbds:	Unknown: Dependent on hypervisor status
Vulnerability Tsa:	Not affected

```
Vulnerability Tsx async abort:      Not affected
Vulnerability Vmscape:              Not affected
```

===== Memory Information =====

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	1.3Gi	612Mi	33Mi	2.2Gi	2.5Gi
Swap:	3.8Gi	120Ki	3.8Gi			

===== Operating System Information =====

```
Linux lizihan-VMware-Virtual-Platform 6.17.0-19-generic #19~24.04.2-Ubuntu SMP PREEMPT_DYNAMIC
```

===== GCC Version =====

```
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

===== Python Version =====

```
Python 3.12.3
```

Appendix B. Python Running Output

===== Correctness Verification =====

```
A =
[1, 2]
[3, 4]
B =
[5, 6]
[7, 8]
Python result C = A x B:
[19, 22]
[43, 50]
Expected result:
[19, 22]
[43, 50]
Verification result: correct
```

===== Python Execution Time Test =====

```
50 x 50 matrix multiplication time: 0.009272 seconds
100 x 100 matrix multiplication time: 0.077458 seconds
150 x 150 matrix multiplication time: 0.264163 seconds
```

Appendix C. Python Source Code

```
import time
import random

def matmul(A, B):
    """
    Implement matrix multiplication  $C = A \times B$  using three nested loops.
    A is an  $n \times m$  matrix, B is an  $m \times p$  matrix, and C is an  $n \times p$  matrix.
    """
    n = len(A)
    m = len(A[0])
    p = len(B[0])

    C = [[0 for _ in range(p)] for _ in range(n)]

    for i in range(n):
        for j in range(p):
            total = 0
            for k in range(m):
                total += A[i][k] * B[k][j]
            C[i][j] = total

    return C

def print_matrix(M):
    for row in M:
        print(row)

def verify():
    A = [
        [1, 2],
        [3, 4]
    ]

    B = [
        [5, 6],
        [7, 8]
    ]

    expected = [
        [19, 22],
        [43, 50]
```

```

]

result = matmul(A, B)

print("A =")
print_matrix(A)

print("B =")
print_matrix(B)

print("Python result C = A x B:")
print_matrix(result)

print("Expected result:")
print_matrix(expected)

if result == expected:
    print("Verification result: correct")
else:
    print("Verification result: incorrect")

def performance_test(size):
    A = [[random.randint(0, 9) for _ in range(size)] for _ in range(size)]
    B = [[random.randint(0, 9) for _ in range(size)] for _ in range(size)]

    start = time.time()
    matmul(A, B)
    end = time.time()

    return end - start

if __name__ == "__main__":
    print("==== Correctness Verification =====")
    verify()

    print("\n==== Python Execution Time Test =====")
    for size in [50, 100, 150]:
        t = performance_test(size)
        print(f"{size} x {size} matrix multiplication time: {t:.6f} seconds")

```